# Übung CHD2 - Chip-Design: Simulation, SS2026
# Homework 1

## Abgabe elektronisch bis 24.4.2026, 23:59 Uhr

**Bitte beachte für die Abgabe UNBEDINGT:**

1. Im E-Learning-Kurs der Übung befinden sich ergänzende VHDL-Dateien. In den Header dieser Dateien sind Vorname und Name einzutragen.
2. Die Ausarbeitung der Aufgabenstellungen ist in diesen Dateien abzugeben. Dabei darf nur der gekennzeichnete Bereich editiert werden (= der Bereich zwischen den „=“-Zeilen).
3. Nur syntaktisch fehlerfreie Dateien (VHDL-93 bzw. VHDL2002) werden angenommen.
4. Die Funktionalität der Ausarbeitungen ist mittels der vorgegebenen Testfälle zu überprüfen.
5. Die vorgegebenen Typen sind **unbedingt** zu verwenden.
6. Bei der Simulation werden Text-Dateien der Testergebnisse im Startverzeichnis des Simulators erstellt. Diese dienen zur Überprüfung der Korrektheit.
7. **Abgabe (elektronisch)**: **Die editierten VHDL-Dateien, die entsprechenden PDF-Dateien und die erzeugten Textdateien sind in ein flaches Verzeichnis zu stellen**. (**KEINE Unterverzeichnisse anlegen**, Name des Verzeichnisses: <Nachname>_HW1). Daraus ist ein ZIP-Archiv (**KEIN RAR!**) zu erstellen, z.B. Mueller_HW1.zip. Die Abgabe erfolgt über das E-Learning-System.
8. **Wichtig**: **Es sind ausnahmslos die Attribute** (siehe Hinweise am Ende) **zu verwenden**. Die Verwendung von Zahlen und Konstanten für Vektorgrenzen im Code führt zu Abzügen.
9. **Extrem wichtig**: Die PDF-Dateien müssen gut lesbar sein. Achtung bei Zeilenumbrüchen!

### Aufgabe 1 – Binär-zu-Integer-Konvertierung

Es ist eine *function* `BinToInt` zu implementieren, die Bitvektoren beliebiger Länge (*x* downto 0) in einen Integer-Wert umrechnet und zurückgibt. Als zusätzlicher Eingabeparameter ist ein Schalter (Typ: `boolean`) vorzusehen. Mit dem Wert `true` wird der Eingabevektor als Darstellung im Zweierkomplement (*signed*) interpretiert. Der Parameterwert `false` kennzeichnet eine vorzeichenlose Dualzahl (*unsigned*). Beobachte und überprüfe die Testfälle im *Waveform*-Fenster und in der Log-Datei.

### Aufgabe 2 – Shift-Operationen

Es sind die Schiebeoperationen `sll` (shift-left logical), `srl` (shift-right logical), `sra` (shift-right arithmetic) und `ror` (rotate right) als *procedures* zu implementieren. Diese *procedures* sollen dieselben Ergebnisse liefern wie die entsprechenden VHDL-Operatoren.
So soll z.B. für die VHDL-Operation `sll` eine *procedure* `my_sll` implementiert werden, die folgende Schnittstelle besitzen muss:

```
procedure my_sll (vector   : in  aBVector;
                  shamt    : in  ashamt;
                  result   : out aBVector)
```

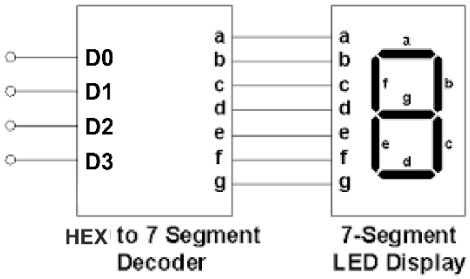
Der Typ `aBVector` ist ein *subtype* des Typs `bit_vector` mit einem ***to-Indexrange***. Abhängig von den Indexgrenzen dieses Typs ist die Anzahl der schiebbaren Positionen durch den Typ `ashamt` (Shift Amount) begrenzt.

Implementiere die Funktionen a) `my_sll` b) `my_srl` c) `my_sra` d) `my_ror`. Teste diese Funktionen mit dem vorgegeben Vektortyp `aBVector` (= `bit_vector (7 to 22)`). Beobachte und überprüfe die Testfälle im *Waveform*-Fenster und in der Log-Datei.

- Es sind ausschließlich die Attribute auf Bit-Vektoren (und keine fixen Werte/Indexgrenzen) zu verwenden. Die implementierten *procedures* müssen für jeden *to-Indexrange* – also z.B. auch für `37 to 103` – funktionieren.

## Aufgabe 3 – Hex-zu-Siebensegment-Decoder

Erstelle die Funktionen `Hexto7SegA` und `Hexto7SegB`, die beide aus einem Bitvektor der Länge 4 die Eingangswerte für eine Siebensegment-Anzeige (Bitvektor der Länge 7) berechnen.




Es sind die Werte von '0' bis 'F' anzuzeigen. Die Segmente der Anzeige werden mit `'a'` bis `'g'` adressiert.

Der Wert '1' bewirkt ein Aufleuchten des adressierten Segments. Bei der Ziffer '6' soll das Segment 'a' leuchten. Bei der Ziffer 'C' soll ein großes "C" (Segmente d, e, f, a) angezeigt werden.

a) Die Funktion `Hexto7SegA` ist mit einem `array` zu realisieren, in dem die Funktionswerte für die Ansteuerung der Siebensegment-Anzeige abgelegt. Die Funktion greift auf den entsprechenden Index in diesem Array zu.
b) Die Funktion `Hexto7SegA` ist mit einem `case-statement` zu realisieren.

Beobachte und überprüfe die Testfälle im *Waveform*-Fenster und in der Log-Datei.

## Attributes:

(Quelle: Markus Pfaff, Modellieren digitaler Schaltungen mit VHDL, 4. Signals, Processes, der Simulation Cycle und Attributes):

```
signal BinVect : bit_vector (31 downto 0);
BinVect'left = 31,
BinVect'right = 0
BinVect'low = 0,
BinVect'high = 31
BinVect'range = 31 downto 0
BinVect'length = 32
```

**Abgabe:** Verzeichnis <Nachname>_HW1 als ZIP-Archiv <Nachname>_HW1.zip mit den
Dateien: HW1Bsp1.vhd, HW1Bsp1.pdf, logHW1Bsp1.txt,
HW1Bsp2.vhd, HW1Bsp2.pdf, logHW1Bsp2.txt
HW1Bsp3.vhd, HW1Bsp3.pdf, logHW1Bsp3.txt
**Achtung**: Keine kompletten Verzeichnisse (z.B. `sim` oder `work`) abgeben!